

The Up-Side-Down Functor: A Reference Implementation of Smart Functors with Boundary-Visible Stance

Shuhei Ihara

June 22, 2026

Abstract

The Up-Side-Down functor is not a new equality, a new physical law, or a new proof principle. It is a structured functor: a pure functor equipped with an adoptive face, a non-adoptive face, a boundary witness, and a coherence contract. The purpose of this paper is to give a reference implementation of such structured functors and of their safe composition operations. The construction treats the adoptive and non-adoptive regions not as two arbitrary pre-existing categories, but as fibers of a single USD record fibration realized over both a carrier category and a two-sided stance category. In this sense a USD functor is a categorical analogue of a smart pointer: after forgetting the additional fields it is an ordinary functor, while before forgetting it carries explicit bookkeeping for observation, non-adoption, boundary visibility, and typed composition. Six API-level compositions are specified: compression, downward restriction, extension, sideways stabilization, temporal turnover, and upward image. Each operation is presented with its type, existence conditions, mathematical implementation, and philosophical reading. The conservative intent is made explicit: USD records may organize proof search or interpretation, but USD-free conclusions must be justified in the base theory.

Contents

1	Scope and non-claims	2
2	Base data	2
3	The USD record fibration	3
4	USD functors as smart functors	5
5	Morphisms and the category of USD functors	7
6	The six composition APIs	8
6.1	C-composition: compression	8
6.2	D-composition: downward restriction	8
6.3	E-composition: extension by a cut vertex	9
6.4	S-composition: sideways shared stabilization	9
6.5	T-composition: temporal turnover	10
6.6	U-composition: upward image	11
6.7	C-D-E and S-T-U reading order	11
7	S-molecules and residue-stable sideways composites	11
8	A syntactic reference implementation	14

9	Worked toy model	15
10	Relativistic test model	16
11	Reviewer-facing checklist	17
12	Open problems	17
13	Conclusion	18

1 Scope and non-claims

This draft responds to a common failure mode in early USD notation: terms such as relation, boundary, observation, adoption, refusal, gap, and turn can sound like explanatory prose rather than mathematical data. The present version therefore treats USD theory as a typed reference implementation. The reader should not be asked to infer the missing interfaces. They are made explicit.

Remark 1.1 (What this paper does not claim). This paper does not claim that USD notation proves a new equality, a new inequality, a physical theorem, or a computational separation. It does not identify boundary-visible connection with equality. It does not identify non-adoption with ordinary negation. It does not claim that the relativistic discussion below proves anything about special or general relativity. The relativistic section is a test model for the vocabulary of observers, frames, local restrictions, and temporal records.

Remark 1.2 (What this paper does claim). This paper claims that one can specify a category of smart functors whose objects are ordinary functors equipped with additional USD structure, and that the main USD composition operations can be given as typed operations using standard categorical vocabulary: pushouts or gluing for compression, pullback or reindexing for downward composition, factorization for extension, fixed points for sideways stabilization, involution plus sequence reversal for temporal turnover, and Kan-style images for upward composition.

The intended comparison is with a smart pointer in programming. A raw pointer stores an address. A smart pointer stores an address together with ownership, lifetime, and release discipline. Likewise, a raw functor stores functorial action. A USD functor stores functorial action together with an adoptive face and a non-adoptive face that are stance-lifts of the same carrier, together with a boundary witness and a coherence contract. Forgetting these fields returns the raw functor. Keeping them enables safe composition of records without silently turning records into proof steps.

2 Base data

This section fixes the data that later sections use. The goal is to avoid introducing objects such as `Rel`, `BVis`, `Obs`, or `NonAdopt` as undefined prose.

Assumption 2.1 (Context category). Let $\mathcal{P}$ be a small category. Its objects are positions, contexts, observers, stages, regions, or local situations. A morphism $\phi : p \rightarrow q$ is an admissible transition of context. In concrete models $\mathcal{P}$ may be a poset, a site, or a category of local charts.

Assumption 2.2 (Indexed entity category). Let

$$E : \mathcal{P}^{\text{op}} \rightarrow \mathbf{Cat}$$

be an indexed category. For each context p the category $E(p)$ contains the entities, expressions, records, or local objects available at p . For each transition $\phi : p \rightarrow q$, the reindexing functor $E(\phi) : E(q) \rightarrow E(p)$ gives the local view of a q -object at p .

Construction 2.3 (Situating occurrence category). The category of situated occurrences is the Grothendieck construction

$$\text{Occ}(E) := \int_{\mathcal{P}} E.$$

An object is a pair (p, x) with $x \in E(p)$. A morphism $(p, x) \rightarrow (q, y)$ is a pair (ϕ, α) where $\phi : p \rightarrow q$ in $\mathcal{P}$ and

$$\alpha : x \rightarrow E(\phi)(y)$$

in $E(p)$. Composition is the usual composition in the Grothendieck construction.

This definition repairs the ambiguity that appears when a set of positions is used while morphisms $p \rightarrow q$ are also required. From this point on, context transitions are morphisms in $\mathcal{P}$.

Assumption 2.4 (Relation witness profunctor). Let $\mathcal{C}$ be a category. A relation witness structure on $\mathcal{C}$ is a profunctor valued in posets,

$$\text{Rel}_{\mathcal{C}} : \mathcal{C}^{\text{op}} \times \mathcal{C} \rightarrow \mathbf{Pos}.$$

An element $r \in \text{Rel}_{\mathcal{C}}(a, b)$ is a witness that a and b are related in the specified sense. Functoriality gives transport of relation witnesses along morphisms of $\mathcal{C}$.

Assumption 2.5 (Boundary visibility). Boundary visibility is a substructure of relation witnesses. We write

$$\text{BVis}_{\mathcal{C}} \hookrightarrow \text{Rel}_{\mathcal{C}}$$

for a subfunctor or indexed subposet. A boundary witness over $r \in \text{Rel}_{\mathcal{C}}(a, b)$ is denoted

$$\beta \in \text{BVis}_{\mathcal{C}}(r).$$

Thus BVis is not a metaphor; it is a typed predicate over relation witnesses.

Remark 2.6 (Minimality of the base). Nothing in the formal development requires Rel to be equality, isomorphism, equivalence, metric proximity, causal accessibility, or logical entailment. Those are model choices. The USD reference implementation only assumes relation witnesses and boundary witnesses.

3 The USD record fibration

The adoptive and non-adoptive sides are not introduced as two unrelated categories. They arise as fibers of a single record fibration.

Definition 3.1 (Stance category). The stance category $\mathbf{St}$ has two distinguished objects

$$U, \quad D,$$

called the adoptive stance and the non-adoptive stance. We assume an involutive equivalence

$$\tau : \mathbf{St} \rightarrow \mathbf{St}, \quad \tau(U) = D, \quad \tau(D) = U, \quad \tau^2 \cong 1_{\mathbf{St}}.$$

The category $\mathbf{St}$ may be taken discrete in the minimal model. In richer models it may contain comparison morphisms between stances.

Definition 3.2 (USD record fibration). A USD record fibration is a functor

$$\pi : \text{Rec}_{\text{USD}} \rightarrow \text{St}.$$

The fiber over U is the adoptive record category

$$\mathcal{U} := \pi^{-1}(U),$$

and the fiber over D is the non-adoptive record category

$$\mathcal{D} := \pi^{-1}(D).$$

The notation $\mathcal{U}$ and $\mathcal{D}$ therefore denotes fibers of one record fibration, not two arbitrary background categories.

Definition 3.3 (Carrier-realized USD record fibration). Fix a carrier category $\mathcal{C}$. A carrier-realized USD record fibration over $\mathcal{C}$ is a USD record fibration together with a realization functor

$$r : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C}$$

and hence a combined projection

$$q = (r, \pi) : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C} \times \text{St}.$$

We require

$$\text{pr}_{\text{St}} \circ q = \pi, \quad \text{pr}_{\mathcal{C}} \circ q = r.$$

Thus every USD record has both a stance and a realized carrier. For $c \in \mathcal{C}$ and $s \in \text{St}$, the notation

$$(\text{Rec}_{\text{USD}})_{(c,s)} := q^{-1}(c, s)$$

denotes the category of stance- s records realizing the carrier datum c . The notation suppresses the dependence of Rec_{USD} on $\mathcal{C}$ when no confusion is possible.

Remark 3.4 (Why realization is required). The projection $\pi : \text{Rec}_{\text{USD}} \rightarrow \text{St}$ separates adoptive from non-adoptive records, but by itself it does not say which ordinary carrier object a record is about. The realization functor $r : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C}$ fills this gap. It prevents the pure carrier, the adoptive face, and the non-adoptive face of a USD functor from becoming three unrelated functors.

Definition 3.5 (Record constructors). The record category contains typed constructors

$$\text{Obs}, \quad \text{Adopt}, \quad \text{NonAdopt}, \quad \text{Bnd}, \quad \text{Collapse},$$

with the following intended types:

$$\begin{aligned} \text{Obs} &: \text{raw datum} \rightarrow \text{Rec}_{\text{USD}}, \\ \text{Adopt} &: \text{Rel}_{\mathcal{C}}(a, b) \rightarrow \mathcal{U}, \\ \text{NonAdopt} &: \text{Rel}_{\mathcal{C}}(a, b) \rightarrow \mathcal{D}, \\ \text{Bnd} &: \text{BVis}_{\mathcal{C}}(r) \rightarrow \text{Rec}_{\text{USD}}, \\ \text{Collapse} &: \text{Rel}_{\mathcal{C}}(a, b) \rightarrow \text{Rec}_{\text{USD}}. \end{aligned}$$

These constructors are part of the syntax of records. A model may interpret them as evidence, annotations, proof-search traces, logs, or semantic states. The realization functor $r : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C}$ records which carrier datum such a constructed record is about; two records may have different stances while realizing the same carrier.

Definition 3.6 (Directed stance notation). Given x, y in a common context, the notation

$$x \rightsquigarrow_{\text{USD}} y$$

means that a boundary-visible connection record from x to y has been constructed. The notation

$$x \dashrightarrow_{\text{USD}} y$$

means that a proposed connection from x to y has been observed but is recorded on the non-adoptive side. Neither notation is equality or ordinary inequality.

4 USD functors as smart functors

We now define the main object. The definition is deliberately close to a reference implementation: a USD functor is a record with fields.

Definition 4.1 (Pure carrier). Let $\mathcal{X}$ and $\mathcal{C}$ be categories. A pure carrier is an ordinary functor

$$F : \mathcal{X} \rightarrow \mathcal{C}.$$

It is the raw functor that remains after forgetting all USD metadata.

Definition 4.2 (Stance-lifts of a carrier). Let $q = (r, \pi) : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C} \times \text{St}$ be a carrier-realized USD record fibration and let $F : \mathcal{X} \rightarrow \mathcal{C}$ be a pure carrier. Write

$$\underline{U}, \underline{D} : \mathcal{X} \rightarrow \text{St}$$

for the constant functors at the two stances. A strict adoptive stance-lift of F is a functor

$$U_F : \mathcal{X} \rightarrow \text{Rec}_{\text{USD}}$$

such that

$$q \circ U_F = (F, \underline{U}),$$

and a strict non-adoptive stance-lift of F is a functor

$$D_F : \mathcal{X} \rightarrow \text{Rec}_{\text{USD}}$$

such that

$$q \circ D_F = (F, \underline{D}).$$

Equivalently, $\pi U_F = \underline{U}$, $\pi D_F = \underline{D}$, and $r U_F = F = r D_F$. In a lax model, these equalities may be replaced by specified comparison 2-cells, but the strict form is the reference implementation used in the main text.

Definition 4.3 (USD functor). A USD functor from $\mathcal{X}$ to $\mathcal{C}$ is a tuple

$$\mathbb{H} = (F, U_F, D_F, \partial_F, \kappa_F)$$

where:

- (i) $F : \mathcal{X} \rightarrow \mathcal{C}$ is a pure carrier functor.
- (ii) $U_F : \mathcal{X} \rightarrow \text{Rec}_{\text{USD}}$ is a strict adoptive stance-lift of F , so $q \circ U_F = (F, \underline{U})$.
- (iii) $D_F : \mathcal{X} \rightarrow \text{Rec}_{\text{USD}}$ is a strict non-adoptive stance-lift of F , so $q \circ D_F = (F, \underline{D})$.

- (iv) ∂_F is boundary data over the common carrier F , relating the two stance-lifts. A minimal implementation treats ∂_F as a family of witnesses

$$\partial_F(x) \in \text{Bnd}_F(U_F(x), D_F(x))$$

natural in $x \in \mathcal{X}$.

- (v) κ_F is a coherence contract governing lifts, boundary data, morphisms, compositions, erasure, and descent.

The critical point is that U_F and D_F do not merely accompany F : they are records over the same realized carrier F .

Definition 4.4 (Coherence contract). For a USD functor $\mathbb{H} = (F, U_F, D_F, \partial_F, \kappa_F)$, the coherence contract is a tuple of obligations

$$\kappa_F = (\kappa_F^{\text{lift}}, \kappa_F^{\partial}, \kappa_F^{\text{mor}}, \kappa_F^{\text{comp}}, \kappa_F^{\text{erase}}, \kappa_F^{\text{desc}}).$$

Here:

- (i) κ_F^{lift} asserts the stance-lift equations $q \circ U_F = (F, \underline{U})$ and $q \circ D_F = (F, \underline{D})$.
- (ii) κ_F^{∂} asserts that ∂_F is defined over the common realized carrier F and is natural in $\mathcal{X}$.
- (iii) κ_F^{mor} specifies what a USD morphism must preserve: carrier, U-face, D-face, boundary witness, and contract data.
- (iv) κ_F^{comp} states the closure obligations for CComp, DComp, EComp, SComp, TComp, and UComp.
- (v) κ_F^{erase} prevents unlicensed USD records from yielding USD-free conclusions after erasure.
- (vi) κ_F^{desc} records which descent licenses, if any, are available for translating records into base assertions.

Thus κ_F is not auxiliary prose; it is the contract that makes the smart functor safe to reference, transform, compose, erase, and descend.

Definition 4.5 (Forgetful functor). There is a forgetful functor

$$|-| : \text{Smart}_{\text{USD}}(\mathcal{X}, \mathcal{C}) \rightarrow [\mathcal{X}, \mathcal{C}]$$

which sends

$$(F, U_F, D_F, \partial_F, \kappa_F) \mapsto F.$$

This is the categorical version of dereferencing the smart functor to its pure carrier.

Remark 4.6 (Why this is not merely gluing two categories). The data $\mathcal{U}$ and $\mathcal{D}$ are not assumed independently and then attached to F . They are stance fibers of the single record fibration $\pi : \text{Rec}_{\text{USD}} \rightarrow \text{St}$, and the same records are realized in the carrier category by $r : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C}$. Consequently the adoptive and non-adoptive faces of a USD functor are stance-lifts over the same pure carrier:

$$q \circ U_F = (F, \underline{U}), \quad q \circ D_F = (F, \underline{D}).$$

The boundary witness ∂_F and coherence contract κ_F therefore relate two readings of one carrier, not two unrelated functors.

Definition 4.7 (Free and anchored USD functors). A free USD functor is a USD functor considered before anchoring it to particular entities. An anchored USD functor is a free USD functor equipped with endpoints or a local occurrence context. We write

$$\text{Anch}_{a,b}(\mathbb{H}) = \mathbb{H}_{a,b} : a \Rightarrow_{\text{USD}} b$$

for anchoring $\mathbb{H}$ between a and b when the required endpoint and boundary data exist.

The distinction is essential. Functions, expressions, programs, formulas, and hypotheses may be free USD functors before they touch any particular pair of entities. Entity-to-entity contact is an anchoring of an already typed USD functor.

5 Morphisms and the category of USD functors

Definition 5.1 (Morphism of USD functors). Let

$$\mathbb{H} = (F, U_F, D_F, \partial_F, \kappa_F), \quad \mathbb{K} = (G, U_G, D_G, \partial_G, \kappa_G)$$

be USD functors from $\mathcal{X}$ to $\mathcal{C}$. A morphism $\eta : \mathbb{H} \rightarrow \mathbb{K}$ consists of:

- (i) a natural transformation $\eta^0 : F \Rightarrow G$ of pure carriers;
- (ii) a natural transformation $\eta^U : U_F \Rightarrow U_G$ in $[\mathcal{X}, \text{Rec}_{\text{USD}}]$ whose realization is the carrier transformation and whose stance component is fixed:

$$q\eta^U = (\eta^0, \text{id}_{\underline{U}});$$

- (iii) a natural transformation $\eta^D : D_F \Rightarrow D_G$ in $[\mathcal{X}, \text{Rec}_{\text{USD}}]$ whose realization is the carrier transformation and whose stance component is fixed:

$$q\eta^D = (\eta^0, \text{id}_{\underline{D}});$$

- (iv) compatibility squares showing that η^U and η^D preserve the boundary data ∂_F and ∂_G over η^0 ;
- (v) preservation of the coherence contracts, in particular κ^{lift} , κ^{∂} , and κ^{erase} .

Proposition 5.2 (Category of USD functors). *For fixed $\mathcal{X}$ and $\mathcal{C}$, USD functors and their morphisms form a category, denoted*

$$\text{Smart}_{\text{USD}}(\mathcal{X}, \mathcal{C}),$$

provided the total record category admits vertical composition of q -compatible natural transformations and the coherence contracts are closed under identity and composition.

Proof. Identities are inherited componentwise from the functor category and the record category. Composition is componentwise and remains compatible with q because carrier and stance components compose in $\mathcal{C} \times \text{St}$. The boundary compatibility and contract preservation conditions are stable by the assumed closure of contracts. The forgetful projection to $[\mathcal{X}, \mathcal{C}]$ is functorial by construction. $\square$

This proposition is intentionally modest. It is not advertised as a deep category-theoretic theorem. It is the sanity check that the reference implementation has the shape of a category.

6 The six composition APIs

The following six operations are specified as APIs. Each operation has a mathematical implementation and a natural-language reading. None is an ordinary proof rule for equality, inequality, or physical truth.

Assumption 6.1 (Common closure invariant for composition). Every composition API below is required to preserve carrier-realized stance-lifts. If the output has carrier $F' : \mathcal{X}' \rightarrow \mathcal{C}$, then its output faces must satisfy

$$q \circ U_{F'} = (F', \underline{U}), \quad q \circ D_{F'} = (F', \underline{D}).$$

This is the operational content of κ^{comp} : composition may change the carrier, boundary, or indexing shape, but it may not detach U-face or D-face records from their carrier realization.

6.1 C-composition: compression

API 6.2 (C-composition). **Input.** Two free USD functors

$$\mathbb{H} : \mathcal{X} \rightarrow \mathcal{C}, \quad \mathbb{K} : \mathcal{Y} \rightarrow \mathcal{C},$$

together with an interface span

$$\mathcal{X} \xleftarrow{i} \mathcal{I} \xrightarrow{j} \mathcal{Y}$$

and interface compatibility data between $i^*\mathbb{H}$ and $j^*\mathbb{K}$.

Existence condition. The pushout

$$\mathcal{X} \sqcup_{\mathcal{I}} \mathcal{Y}$$

exists in **Cat**, and the carrier and record data glue along the interface in a way compatible with q . In particular, the glued U-face and D-face must be stance-lifts of the glued carrier.

Output. A free USD functor

$$\text{CComp}_{\mathcal{I}}(\mathbb{H}, \mathbb{K}) : \mathcal{X} \sqcup_{\mathcal{I}} \mathcal{Y} \rightarrow \mathcal{C}.$$

Mathematically, C-composition is gluing or compression by a pushout of domains plus compatible gluing of the carrier, adoptive face, non-adoptive face, and boundary witness. It compresses two freely available USD functors into one referenceable USD functor. It does not assert that the two inputs were identical; the joint boundary record retains the old boundary data and the new interface discrepancy.

A minimal boundary policy is

$$\partial_{\text{CComp}(\mathbb{H}, \mathbb{K})} = \partial_{\mathbb{H}} \oplus \partial_{\mathbb{K}} \oplus \Delta_{\mathcal{I}}(\mathbb{H}, \mathbb{K}),$$

where $\Delta_{\mathcal{I}}$ is the interface discrepancy supplied by the model.

6.2 D-composition: downward restriction

API 6.3 (D-composition). **Input.** A context morphism

$$\rho : \mathcal{L} \rightarrow \mathcal{G}$$

and a USD functor over the global context $\mathcal{G}$.

Mathematical implementation. D-composition is reindexing or pullback:

$$\text{DComp}_{\rho} = \rho^* : \text{Smart}_{\text{USD}}(\mathcal{G}, \mathcal{C}) \rightarrow \text{Smart}_{\text{USD}}(\mathcal{L}, \mathcal{C}).$$

Output. A local USD functor $\rho^*\mathbb{H}$.

Lift condition. If $\mathbb{H}$ has carrier F , then the output has carrier ρ^*F , and its faces are the reindexed stance-lifts satisfying

$$q(\rho^*U_F) = (\rho^*F, \underline{U}), \quad q(\rho^*D_F) = (\rho^*F, \underline{D}).$$

The natural-language reading is: a large observed USD functor is lowered to a local context. The operation is not a proof that the local context satisfies the global claim. It is the typed act of reading the global record through a local index.

6.3 E-composition: extension by a cut vertex

API 6.4 (E-composition). **Input.** An anchored USD functor

$$\mathbb{H}_{a,b} : a \Rightarrow_{\text{USD}} b$$

and a proposed cut vertex m .

Existence condition. There is factorization data in the relevant arrow category, together with a comparison 2-cell

$$\varepsilon_m : \mathbb{H}_{a,m} \circ \mathbb{H}_{m,b} \Rightarrow \mathbb{H}_{a,b}$$

or the reverse comparison, depending on the model.

Output. A subdivided anchored record

$$\text{EComp}_m(\mathbb{H}_{a,b}) = (\mathbb{H}_{a,m}, \mathbb{H}_{m,b}; \varepsilon_m).$$

E-composition extends one anchored USD functor by adding an internal observation point. It should be read as subdivision or refinement, not as a proof that the original record is strictly equal to the composite.

6.4 S-composition: sideways shared stabilization

S-composition is the operation most in need of explicit existence conditions. We therefore specify the gap space before defining the operation.

Assumption 6.5 (Gap lattice). For a pair of compatible USD functors $\mathbb{H}, \mathbb{K}$, assume a complete lattice

$$\Lambda_{\mathbb{H}, \mathbb{K}}$$

of possible gap values. The order $\leq$ means “no more discrepant than” or any chosen model-specific refinement order.

Assumption 6.6 (Gap update). A sideways interaction supplies a monotone update map

$$\Phi_{\mathbb{H}, \mathbb{K}} : \Lambda_{\mathbb{H}, \mathbb{K}} \rightarrow \Lambda_{\mathbb{H}, \mathbb{K}}.$$

The value $\Phi(\Delta)$ is the new discrepancy after both sides have incorporated the current discrepancy into their non-adoptive records.

API 6.7 (S-composition). **Input.** A pair of compatible USD functors $\mathbb{H}, \mathbb{K}$ and a monotone gap update map $\Phi_{\mathbb{H}, \mathbb{K}}$ on a complete lattice of gaps.

Existence condition. A fixed point

$$\Delta^* = \Phi_{\mathbb{H}, \mathbb{K}}(\Delta^*)$$

is selected. Existence follows, for example, from the Knaster-Tarski fixed point theorem when the stated monotonicity and completeness conditions hold.

Output. A sideways stabilized USD record

$$\text{SComp}(\mathbb{H}, \mathbb{K}; \Delta^*).$$

Natural-language reading: two USD functors do not eliminate their gap. They update their non-adoptive faces until the same gap is shared as a stable discrepancy. The completion condition is not $\Delta^* = 0$ but shared stability.

6.5 T-composition: temporal turnover

Definition 6.8 (Sequence category). Let $\text{Seq}(\text{Smart}_{\text{USD}})$ be the category of finite composable sequences of USD functors. An object is a finite chain

$$\mathcal{H} = (\mathbb{H}_1, \dots, \mathbb{H}_n).$$

Definition 6.9 (U/D flip). Assume that the stance involution $\tau : \text{St} \rightarrow \text{St}$ lifts to a carrier-preserving record involution

$$\bar{\tau} : \text{Rec}_{\text{USD}} \rightarrow \text{Rec}_{\text{USD}}, \quad r\bar{\tau} = r, \quad \pi\bar{\tau} = \tau\pi.$$

The induced operation

$$\text{Flip}_{U,D} : \text{Smart}_{\text{USD}} \rightarrow \text{Smart}_{\text{USD}}$$

sends

$$(F, U_F, D_F, \partial_F, \kappa_F) \mapsto (F, \bar{\tau}D_F, \bar{\tau}U_F, \partial_F^\tau, \kappa_F^\tau).$$

The pure carrier is not altered. The lifted faces satisfy

$$q(\bar{\tau}D_F) = (F, \underline{U}), \quad q(\bar{\tau}U_F) = (F, \underline{D}),$$

so T-composition turns the record upside down without touching the realized carrier.

API 6.10 (T-composition). **Input.** A finite USD functor chain

$$\mathcal{H} = (\mathbb{H}_1, \dots, \mathbb{H}_n).$$

Mathematical implementation.

$$\text{TComp}(\mathcal{H}) := \text{Rev}(\text{Flip}_{U,D}(\mathbb{H}_1), \dots, \text{Flip}_{U,D}(\mathbb{H}_n)),$$

so that

$$\text{TComp}(\mathcal{H}) = (\mathbb{H}_n^\tau, \dots, \mathbb{H}_1^\tau).$$

Output. A temporally turned-over USD record chain.

T-composition touches no entity. It reverses the order of the record chain and applies the carrier-preserving stance turnover to every adoptive and non-adoptive face. A chain that was read forward as an identification proposal may be re-read backward as a non-adoption record, but this is a transformation of records, not a proof of ordinary inequality.

If no information is lost, **TComp** is involutive up to record equivalence:

$$\text{TComp}^2 \cong 1.$$

If compression, stabilization, or descent has discarded information, the weaker USD-level comparison

$$\text{TComp}^2(\mathcal{H}) \rightsquigarrow_{\text{USD}} \mathcal{H}$$

may be all that remains.

6.6 U-composition: upward image

D-composition has two common adjoints when the relevant Kan extensions exist. We keep the existential and universal readings separate.

API 6.11 (U-composition). Let $\rho : \mathcal{L} \rightarrow \mathcal{G}$ be a context morphism.

Existential upward composition. When the left Kan extension exists, define

$$\text{UComp}_\rho^\exists := \Sigma_\rho := \text{Lan}_\rho : \text{Smart}_{\text{USD}}(\mathcal{L}, \mathcal{C}) \rightarrow \text{Smart}_{\text{USD}}(\mathcal{G}, \mathcal{C}).$$

Then

$$\Sigma_\rho \dashv \rho^*.$$

Universal upward composition. When the right Kan extension exists, define

$$\text{UComp}_\rho^\forall := \Pi_\rho := \text{Ran}_\rho : \text{Smart}_{\text{USD}}(\mathcal{L}, \mathcal{C}) \rightarrow \text{Smart}_{\text{USD}}(\mathcal{G}, \mathcal{C}).$$

Then

$$\rho^* \dashv \Pi_\rho.$$

Smart-lift condition. These Kan extensions are admitted as U-compositions only when they lift from the pure carrier category to $\text{Smart}_{\text{USD}}$. For example, if $\Sigma_\rho \mathbb{H}$ has carrier $\text{Lan}_\rho F$, then its faces must be records over that carrier:

$$q \circ U_{\text{Lan}_\rho F} = (\text{Lan}_\rho F, \underline{U}), \quad q \circ D_{\text{Lan}_\rho F} = (\text{Lan}_\rho F, \underline{D}),$$

and similarly for Π_ρ and $\text{Ran}_\rho F$. The existence of the ordinary Kan extension of F is therefore not enough; the USD record and contract data must lift with it.

If these lifted Kan extensions do not exist, the relevant U-composition is undefined in the smart category even when the carrier Kan extension exists.

The natural-language reading is that a local USD functor is pushed upward as a global candidate record. The existential version says that some local contribution is imaged globally; the universal version says that a local condition is extended as a global constraint when possible.

6.7 C-D-E and S-T-U reading order

The six operations have two useful reading sequences:

$$\text{CComp} \longrightarrow \text{DComp} \longrightarrow \text{EComp}$$

and

$$\text{SComp} \longrightarrow \text{TComp} \longrightarrow \text{UComp}.$$

The first sequence compresses free records, lowers them, and subdivides anchored local records. The second stabilizes a shared gap, turns the record chain upside down, and then images the result upward.

7 S-molecules and residue-stable sideways composites

The preceding APIs define how USD records may be composed. We now record one derived notion that will be useful in examples: an *S*-molecule. The term is intentionally internal to USD theory. It does not mean a chemical molecule unless a separate physical realization or descent license is supplied.

Definition 7.1 (U/D interlock). Let A and B be compatible USD records or USD functors. A U/D interlock between A and B is boundary-visible data witnessing that the adoptive face of each side can be read against the non-adoptive face of the other side. In notation, this is recorded as

$$U_A \rightsquigarrow_{\text{USD}} D_B, \quad U_B \rightsquigarrow_{\text{USD}} D_A,$$

together with the relevant boundary witnesses and coherence data. The symbol $\rightsquigarrow_{\text{USD}}$ is not equality. It records a boundary-visible connection between stance-lifted records over their realized carriers.

Definition 7.2 (S-molecule). Let A and B be compatible USD records or USD functors. Assume a complete gap lattice

$$\Lambda_{A,B}$$

and a monotone sideways update map

$$\Phi_{A,B} : \Lambda_{A,B} \rightarrow \Lambda_{A,B}.$$

An S -molecule between A and B is a sideways-stabilized record

$$\text{SMol}(A, B; \Delta^*)$$

consisting of a fixed point

$$\Delta^* = \Phi_{A,B}(\Delta^*)$$

together with a U/D interlock between A and B , such that the fixed gap Δ^* is preserved by the U-face, D-face, boundary witness, and coherence contract of the resulting sideways composite.

Thus an S -molecule is not the collapse of A and B into one identity. It is a USD composite in which two distinguishable sides keep a shared stable discrepancy and become observable as one composite record.

Remark 7.3 (Meaning of molecule). The word “molecule” is used here by analogy with stable composite entities. A physical covalent molecule, a polymer, a software package dependency cluster, or a social institution may be modeled as an S -molecule only after specifying an interpretation of A , B , the gap lattice, the update map, and the relevant boundary witnesses. Without such an interpretation, $\text{SMol}(A, B; \Delta^*)$ is only a USD record.

Definition 7.4 (Positive-negative exchange). Let $\star$ be one of the USD composition APIs, whenever its input data are compatible. A positive-negative exchange datum, or PN-exchange datum, for a composition instance consists of an estimated gain

$$\widehat{G}_\star$$

and an estimated loss

$$\widehat{L}_\star.$$

The estimated gain is the portion of the composition adopted by the main output record. The estimated loss is not erased. It is recorded as non-adopted residue attached to the output.

Accordingly, a residue-aware USD composition returns not only a main composite $\mathbb{M}_\star$, but also a second USD functor

$$\mathbb{W}_\star : \mathbb{M}_\star \Rightarrow_{\text{USD}} \mathbb{M}_\star,$$

called the self-anchored residue functor. Thus the output shape is

$$\star(\text{input data}) = (\mathbb{M}_\star, \mathbb{W}_\star).$$

The residue functor stores the loss, discrepancy, non-adoption, delay, and boundary-visible waste that the main composite does not adopt.

Definition 7.5 (PN-conservation). Let

$$\text{Qty} : \text{Smart}_{\text{USD}}(\mathcal{X}, \mathcal{C}) \rightarrow \mathcal{R}$$

be a USD quantity valuation into an ordered commutative monoid $\mathcal{R}$. A PN-exchange datum is conservative when

$$\text{Qty}(\text{input records}) = \text{Qty}(\mathbb{M}_\star) + \text{Qty}(\mathbb{W}_\star).$$

Thus USD quantity is not destroyed by composition. What is not adopted by the main composite remains as a self-anchored residue functor.

Remark 7.6 (Why residue is part of the composition). Without the residue functor, a USD composition may look as if the unadopted part of the interaction has disappeared. The PN-exchange policy prevents this. It makes every loss or mismatch accountable either inside the main composite or inside its self-anchored residue. This is consistent with the conservative design of USD theory: unadopted records are not silently converted into USD-free conclusions.

Definition 7.7 (UD-spin). Let

$$\mathbb{M} = \text{SMol}(A, B; \Delta^*)$$

be an S -molecule, and let ξ be an external perturbation, force, load, attack, or environmental input applied to $\mathbb{M}$. A UD-spin generated by ξ is a finite or eventually stationary internal update sequence

$$S_0 \rightarrow S_1 \rightarrow \cdots \rightarrow S_n$$

in which the perturbation circulates between the U-face and D-face of $\mathbb{M}$ before being absorbed, recorded as residue, or ejected through a boundary-visible residue functor.

UD-spin is a USD bookkeeping operation. It is not physical spin unless a separate physical model identifies it with a physical spin-like quantity.

Definition 7.8 (Divergence, ejection, and spin termination). Let ω denote a UD-spin speed for an S -molecule $\mathbb{M}$. Let

$$\text{Div}_{\mathbb{M}}(\omega)$$

be the amount of perturbation that $\mathbb{M}$ can internally dissipate or diverge at spin speed ω , and let

$$\text{Eject}_{\mathbb{M}}(\omega)$$

be the amount that can be ejected through boundary-visible residue at that speed. If λ_ξ denotes the incoming perturbation rate, then the basic stability condition is

$$\lambda_\xi \leq \text{Div}_{\mathbb{M}}(\omega) + \text{Eject}_{\mathbb{M}}(\omega).$$

When this condition holds and the induced UD-spin terminates, the S -molecule returns to a state that is observationally equivalent, at the chosen boundary resolution, to the original composite record.

Definition 7.9 (Critical spin speed). The critical spin speed of an S -molecule $\mathbb{M}$ is the boundary spin speed at which the incoming perturbation rate begins to exceed the sum of the divergence capacity and ejection capacity:

$$\lambda_\xi > \text{Div}_{\mathbb{M}}(\omega) + \text{Eject}_{\mathbb{M}}(\omega).$$

Beyond this boundary, residue accumulates faster than it can be processed. If R_t denotes accumulated residue, one may record the update schematically as

$$R_{t+1} = R_t + \lambda_\xi - (\text{Div}_{\mathbb{M}}(\omega) + \text{Eject}_{\mathbb{M}}(\omega)).$$

When the residue exceeds the tolerance allowed by the coherence contract, the S -molecule may break, dissociate, reconfigure, or cease to be observable as one composite record.

Definition 7.10 (Observed UD-spin speed). The internal UD-spin speed of an S -molecule and the externally observed UD-spin speed need not coincide. We write

$$\omega_{\mathbb{M}}^{\text{int}}$$

for the internal spin speed and

$$\omega_{\mathbb{M}}^{\text{Obs}}$$

for the boundary-visible observed spin speed. If $\alpha_{\mathbb{M}}$ denotes observational resolution or accuracy, then one may model the observed speed as

$$\omega_{\mathbb{M}}^{\text{Obs}} = \alpha_{\mathbb{M}} \cdot \omega_{\mathbb{M}}^{\text{eff}}.$$

For a long serial chain of S -molecules, observational resolution may decrease as the chain length increases. In such models the observed UD-spin speed is allowed to remain constant or decrease, even if local internal spin speeds do not decrease.

Definition 7.11 (Non-observable pending wait). During the formation or perturbation of an S -molecule, internal non-equivalent exchanges need not be externally observable. Let σ_A be the time at which an internal accumulation begins in A , let θ_A be the time at which that accumulation becomes boundary-visible, and let $\tau_{A \rightarrow B}$ be the time at which the corresponding exchange is implemented toward B . The non-observable pending wait from A to B is

$$T_{A \rightarrow B}^{\text{NP}} = \tau_{A \rightarrow B} - \theta_A.$$

Here NP means “non-observable pending” and is unrelated to the complexity class NP.

Definition 7.12 (S-polymer). An S -polymer is a finite or indexed composite of S -molecules

$$\text{SMol}_1 \text{SComp SMol}_2 \text{SComp} \cdots \text{SComp SMol}_n$$

which is externally observable as one higher-order USD entity at a chosen boundary resolution. An S -polymer may model a physical material, a software dependency cluster, an institution, or another composite system, provided that a suitable realization of the USD records is supplied.

Remark 7.13 (S-molecules and physical examples). A covalent bond may be used as an intuitive model of an S -molecule: two atoms remain distinguishable while forming a stable shared state through electron-density-mediated interaction. This comparison is only a model reading. USD theory does not redefine covalent bonding, molecular physics, software dependency, or any other base-domain concept. To obtain a USD-free physical conclusion, the relevant descent license must still be supplied in the base theory.

8 A syntactic reference implementation

This section records the minimum syntax needed for a conservative USD calculus. It is intentionally austere.

Definition 8.1 (Base language). Let $\mathcal{L}_0$ be a base formal language with judgments

$$\Gamma \vdash_0 A.$$

The base language may be first-order logic, type theory, an internal language of a category, or another fixed formal system.

Definition 8.2 (USD extension). The USD calculus USDCalc extends $\mathcal{L}_0$ with record terms:

$$\text{conn}(x, y; r, \beta), \quad \text{nonadopt}(x, y; r, \beta), \quad \text{hand}(F, U, D, \partial, \kappa),$$

and with record-forming operations

$$\text{CComp}, \text{DComp}, \text{EComp}, \text{SComp}, \text{TComp}, \text{UComp}.$$

The USD extension does not add a base rule allowing a record term alone to prove a USD-free formula.

Definition 8.3 (Descent license). A descent license is an explicit rule instance

$$\text{IdLic}(R, A)$$

that permits a USD record R to be translated into a base assertion A in $\mathcal{L}_0$. Examples include a definition, an isomorphism, a proved equivalence, a semantic interpretation, or a verified coercion.

Definition 8.4 (Erasure). The erasure translation

$$|-| : \text{USDCalc} \rightarrow \mathcal{L}_0$$

acts as identity on base terms and formulas and removes pure USD records unless a descent license supplies a base formula.

Theorem 8.5 (Conservative design theorem). *Assume that every USD inference rule is record-forming, record-transforming, or explicitly licensed by a descent rule. If*

$$\Gamma \vdash_{\text{USDCalc}} A$$

and A is USD-free, then the derivation of A uses only base rules and licensed descents. Consequently

$$|\Gamma| \vdash_0 A$$

provided each used descent license is valid in the base system.

Proof. By induction on the given derivation. Base rules translate to base rules. Record-forming and record-transforming USD rules have USD-record conclusions and therefore cannot be the final step of a USD-free formula. If a USD record is used to produce a USD-free conclusion, the rule must be a licensed descent by hypothesis; replacing that step with its base license yields a derivation in $\mathcal{L}_0$. The induction removes all unlicensed USD bookkeeping from the derivation of A . $\square$

This theorem is deliberately conditional. It states the safety contract of the reference implementation. A stronger metatheorem for a particular formal calculus would require a fully specified proof system, but this condition is already enough to prevent USD records from acting as hidden proof rules.

9 Worked toy model

Example 9.1 (Poset record model). Let $\mathcal{C}$ be a poset regarded as a category. Let

$$\text{Rel}_{\mathcal{C}}(a, b)$$

be the truth value of $a \leq b$. Let $\mathbf{BVis}$ select those comparable pairs for which a chosen boundary label is present. Let $\mathbf{Rec}_{\text{USD}}$ be the product category

$$\mathcal{C} \times \mathbf{St}$$

with projection to $\mathbf{St}$. The realization projection is

$$r = \text{pr}_{\mathcal{C}} : \mathcal{C} \times \mathbf{St} \rightarrow \mathcal{C},$$

so the combined projection is

$$q = (r, \pi) : \mathcal{C} \times \mathbf{St} \rightarrow \mathcal{C} \times \mathbf{St},$$

the identity in this minimal model. Then $\mathcal{U} \cong \mathcal{C} \cong \mathcal{D}$, but the two copies are distinguished as fibers. A USD functor is a functor F together with two stance-lifted copies U_F, D_F over the same F and a boundary label. In this model D-composition is ordinary reindexing of monotone maps, C-composition is pushout of indexing posets when it exists, and T-composition swaps the two copies and reverses a finite chain while preserving realization.

Example 9.2 (Complete lattice gap model). Let the gap lattice be a complete lattice Λ . If a sideways interaction gives a monotone map $\Phi : \Lambda \rightarrow \Lambda$, then the set of fixed points is nonempty by the Knaster-Tarski theorem. Choosing the least fixed point gives a canonical minimal shared discrepancy. Choosing the greatest fixed point gives a maximal conservative discrepancy. The choice is part of the model policy.

10 Relativistic test model

The relativistic material is not an application claiming new physics. It is a test model for the USD vocabulary because relativity naturally involves observers, frames, local restrictions, and time-oriented records. The historical sources are Einstein's 1905 paper on special relativity and his 1916 paper on general relativity [6, 7].

Definition 10.1 (Lorentzian test base). Let (M, g) be a Lorentzian manifold. Let $\mathcal{O}(M, g)$ be a chosen category of local observers or frames. A morphism in $\mathcal{O}(M, g)$ may be a change of frame, inclusion of a local chart, or another chosen admissible comparison of observations.

Example 10.2 (D-composition as local restriction). A global geometric record, such as a metric-dependent record on (M, g) , may be lowered along a local frame map

$$\rho : O \rightarrow M$$

by pullback:

$$\mathbf{DComp}_{\rho}(\mathbb{H}) = \rho^* \mathbb{H}.$$

This is the familiar mathematical shape of restricting global data to a local observer or frame. The USD interpretation only adds adoptive and non-adoptive record faces and boundary witnesses.

Example 10.3 (T-composition as record turnover). A time-oriented chain of observation records may be transformed by $\mathbf{TComp}$ into the reverse-order chain with U and D faces exchanged. This is not asserted to be physical time reversal symmetry. It is a record operation that leaves the entities and geometric carrier untouched while turning the USD bookkeeping upside down.

Remark 10.4 (No hidden physics). If a model uses Lorentz transformations, connections, curvature, or field equations, those structures must be explicitly included in the base category or in the relation witness profunctor. USD functors do not supply them automatically.

11 Reviewer-facing checklist

For reference, the implementation satisfies the following design checks.

1. The context parameter is a category $\mathcal{P}$, not a bare set.
2. Situated occurrences are defined by a Grothendieck construction.
3. Rel is a poset-valued profunctor or indexed predicate.
4. BVis is a typed substructure of Rel .
5. $\mathcal{U}$ and $\mathcal{D}$ are fibers of one fibration $\text{Rec}_{\text{USD}} \rightarrow \text{St}$, and every record is also realized by $r : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C}$.
6. The combined projection $q = (r, \pi) : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C} \times \text{St}$ makes U- and D-faces stance-lifts over one carrier.
7. A USD functor is a tuple $(F, U_F, D_F, \partial_F, \kappa_F)$ where U_F and D_F satisfy $q \circ U_F = (F, \underline{U})$ and $q \circ D_F = (F, \underline{D})$.
8. The coherence contract κ_F is an explicit bundle of lift, boundary, morphism, composition, erasure, and descent obligations.
9. C/D/E/S/T/U operations have stated input types, outputs, and existence conditions.
10. S-composition depends on a complete lattice and a monotone update map, so fixed-point existence is not merely asserted.
11. T-composition is sequence reversal plus a carrier-preserving U/D involution and does not touch entities or carrier realization.
12. USD-free conclusions require base reasoning or an explicit descent license.

12 Open problems

The present draft is a reference implementation rather than a final metatheory. The main open problems are:

1. Refine preferred subclasses of coherence contracts κ_F and prove closure under all six composition APIs in nontrivial models.
2. Compare the carrier-realized USD record fibration with displayed categories, double categories, fibrations of proofs, and decorated functors.
3. Give a fully formal sequent calculus or type theory for USDCalc and prove a cut-elimination or model-theoretic conservativity theorem.
4. Identify useful model policies for selecting fixed points in S-composition.
5. Develop lax stance-lift variants where $q \circ U_F$ and $q \circ D_F$ compare to $(F, \underline{U})$ and $(F, \underline{D})$ rather than strictly equal them.
6. Develop nontrivial test models beyond the poset and relativistic sketches.

13 Conclusion

The central correction in this draft is that USD Functor Theory should be read as a specification of smart functors, not as a metaphorical use of category theory. A USD functor is an ordinary functor with additional record fields: adoptive face, non-adoptive face, boundary witness, and coherence contract. The U and D regions are fibers of one stance-indexed record fibration, and the faces are stance-lifts realized over the same carrier by

$$q = (r, \pi) : \text{Rec}_{\text{USD}} \rightarrow \mathcal{C} \times \text{St}.$$

The six composition operations are typed APIs backed by ordinary categorical constructions or explicit existence assumptions, and each operation is defined only when it preserves the stance-lift invariant through q . The resulting system can guide interpretation and proof search while remaining conservative over the base language for USD-free conclusions.

References

- [1] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, 1971.
- [2] Francis Borceux. *Handbook of Categorical Algebra, Volumes 1–3*. Cambridge University Press, 1994.
- [3] Peter T. Johnstone. *Sketches of an Elephant: A Topos Theory Compendium*. Oxford University Press, 2002.
- [4] Alexander Grothendieck. Revêtements étales et groupe fondamental. In *SGA 1*, Lecture Notes in Mathematics 224, Springer, 1971.
- [5] Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955.
- [6] Albert Einstein. Zur Elektrodynamik bewegter Körper. *Annalen der Physik*, 322(10):891–921, 1905.
- [7] Albert Einstein. Die Grundlage der allgemeinen Relativitätstheorie. *Annalen der Physik*, 354(7):769–822, 1916.